A Project Report on

Inventory Management System

Submitted to

DR. BABASAHEB AMBEDKAR TECHNOLOGICAL UNIVERSITY, LONERE

in partial fulfillment of the requirement for the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE & ENGINEERING

By

Chandravanshi Aditya 106
Kshisagar Rohan 104
[TY-CSE A]

Under the Guidance
of

Ms.M.G.Shelke

(Department of Computer Science and Engineering)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
MAHATMA GANDHI MISSION'S COLLEGE OF ENGINEERING
NANDED (M.S.)

Academic Year 2025-26

Certificate

This is to certify that the project entitled

"Inventory Management System"

being submitted by ***Mr. Chandravanshi aditya*** *and* ***Mr Kshisagar rohan*** *to the Dr. Babasaheb Ambedkar Technological University, Lonere, for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of bonafide work carried out by them under my supervision and guidance. The matter contained in this report has not been submitted to any other university or institute for the award of any degree*.

Ms.M.G.Shelke
Project Guide

Dr. A. M. Rajurkar
H.O.D
Computer Science & Engineering

Dr. G. S. Lathkar
Director
MGM's College of Engineering, Nanded

ACKNOWLEDGEMENT

We are greatly indebted to our seminar guide **Ms.M.G.Shelke** for her able guidance throughout this work. It has been an altogether different experience to work with her and we would like to thank her for her help, suggestions and numerous discussions.

We gladly take this opportunity to thank **Dr.Rajurkar A.M.** (Head of Computer Science & Engineering, MGM's College of Engineering, Nanded).

We are heartily thankful to **Dr. Lathkar G. S.** (Director, MGM's College of Engineering, Nanded) for providing facility during progress of seminar, also for her kindly help, guidance and inspiration.

Last but not least we are also thankful to all those who help directly or indirectly to develop this seminar and complete it successfully.

With Deep Reverence,

Aditya Chandravanshi

Kshisagar rohan

ABSTRACT

Inventory Management System

The Inventory Management System is a web-based application designed to streamline and digitalize essential electoral services, making voter-related processes more efficient, transparent, and accessible for citizens. Built using HTML, CSS, and JavaScript for the frontend and Flask with MySQL for the backend, the system provides a secure and responsive platform that allows users to register as new voters, request corrections or updates to their existing voter details, and track the real-time status of their applications without physically visiting electoral offices. The portal includes a role-based authentication mechanism to differentiate between voter and administrator access, ensuring that only authorized personnel can manage voter records, review submitted applications, and update their verification status. It also incorporates secure data handling practices, such as password hashing and strict database validations, ensuring data reliability and protection throughout the system. The user-friendly interface enhances accessibility across all devices, supporting a seamless experience for users from diverse backgrounds. By reducing manual paperwork, eliminating delays, and promoting digital governance, the Inventory Management System significantly improves the electoral registration process and encourages greater democratic participation. This system demonstrates how technology can strengthen governance by offering convenience, transparency, and efficiency in managing voter services.

TABLE OF CONTENTS

LIST OF FIGURES

Chapter **1**

Introduction and System Overview

This chapter introduces the Inventory Management System, an online system designed to digitalize voter registration and improve the accuracy, security, and efficiency of managing voter information in the electoral process. It outlines the project background, objectives, scope, system users, and key stakeholders involved in implementing and maintaining the portal

1.1 Project Background

The electoral process requires accurate, efficient, and transparent management of voter information. Traditional paper-based systems are prone to errors, duplication, and delays. The Inventory Management System addresses these challenges by digitizing the voter registration process, enabling voters to apply for voter IDs online while providing administrators with tools to efficiently process applications [1].

1.2 Project Objectives

The primary objectives of the Inventory Management System are:

1. **Digitalize Voter Registration**: Enable citizens to register and apply for voter IDs through an online platform
2. **Streamline Application Processing**: Provide administrators with efficient tools to review, approve, or reject voter applications
3. **Maintain Data Integrity**: Implement normalized database design to ensure data accuracy and consistency
4. **Ensure Security**: Protect voter information through encryption, access controls, and input validation
5. **Enable Information Updates**: Allow voters to request updates to their registered information
6. **Generate Official Documentation**: Create and manage election cards for approved voters

1.3 Scope and Boundaries

In Scope:

- Voter registration and application submission
- Admin dashboard and application management
- Election card generation and issuance

- Update request processing
- Role-based access control
- Secure authentication

Out of Scope:

- Actual voting process
- Multi-language support (initial release)
- Mobile native applications
- Third-party integrations
- Advanced analytics and reporting

1.4 System Users

Primary Users:

Voters

- Individuals seeking to register or apply for voter IDs
- Can submit applications, track status, and request information updates
- View issued election cards

Administrators

- Election officials responsible for application review and approval
- Can view applications, add remarks, approve/reject registrations
- Process update requests from voters

System Administrators

- IT personnel managing system deployment and maintenance
- Database administration and backups
- System configuration and security updates

1.5 Stakeholders

- Election Commission authorities
- Voters and citizens
- Election administrators
- Technical development team
- System maintenance staff

Looking ahead, the next chapter will explore the technology stack and system architecture that power the Inventory Management System. It will explain how various software frameworks, database designs, and security measures come together to create a reliable and efficient platform. This technical foundation ensures smooth voter registration, secure application handling, and the trusted generation of election cards, building on the goals and structure introduced here.

Chapter 2

Technology Stack and Architecture

This chapter outlines the technology stack and three-tier architecture powering the Inventory Management System, featuring Flask for lightweight backend logic, MySQL for secure data management, and modern frontend tools like HTML5, CSS3, and JavaScript for responsive user interfaces. It covers key components, request flows, and development requirements to ensure scalability and reliability in voter registration processes.

2.1 Technology Selection

The Inventory Management System employs a modern web development stack optimized for scalability, security, and maintainability [1][2]:

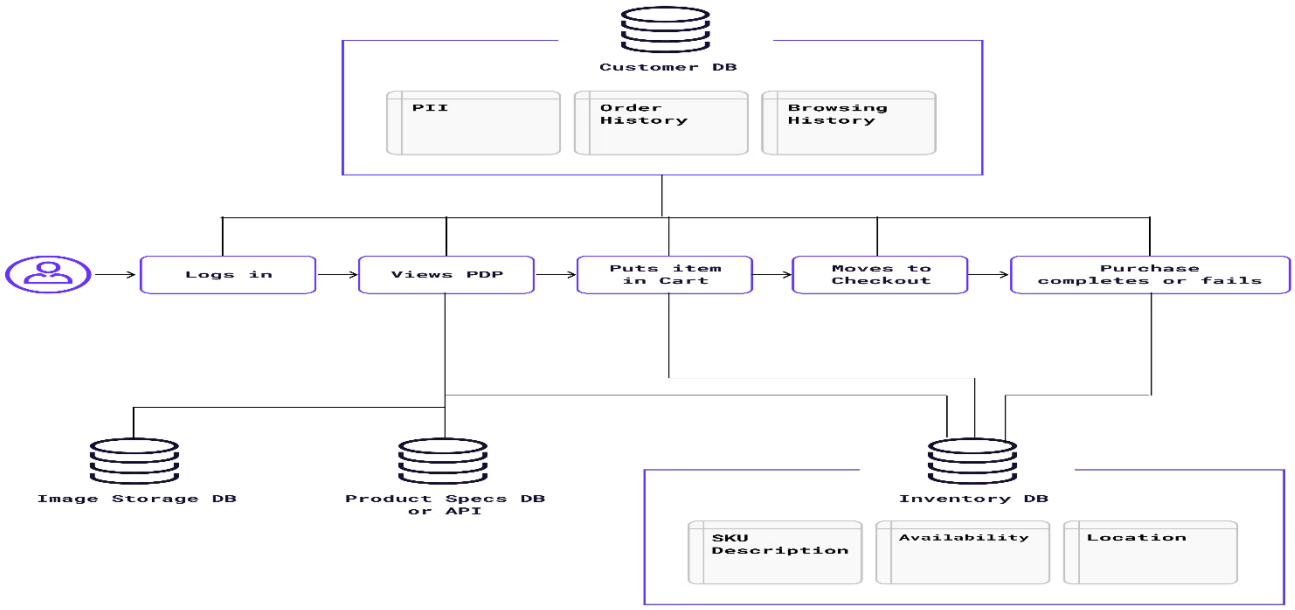

Figure 2.1: Inventory Management System System Architecture Overview

2.2 Backend Technology: Flask

Framework: Flask 2.3.3

Language: Python 3.x

Type: Micro web framework

: Suitable for projects from small to enterprise-scale

- **Community**: Extensive documentation and active community support
- **Python Integration**: Leverages Python's rich ecosystem for data handling

Core Flask Components Used:

- Session management for user authentication
- Jinja2 templating engine for dynamic HTML generation
- Request routing and URL handling
- Cookie and session management
- Error handling and logging
- Application factory patterns for scalability

2.3 Database Technology: MySQL

Database System: MySQL (Relational)

Connector: PyMySQL 1.1.0

Design Approach: Normalized (Third Normal Form - 3NF)

Database Characteristics:

- **Open Source**: Free and widely supported
- **ACID Compliance**: Ensures data consistency and reliability
- **Normalization**: Eliminates data redundancy and maintains referential integrity
- **Scalability**: Handles large datasets efficiently
- **Security**: User authentication and granular permissions

Database Connection Pattern:

DB_CONFIG = {

'host': 'localhost',

'user': 'root',

'password': 'password',

'database': 'voter_portal_db'}

def get_db_connection():

return pymysql.connect(**DB_CONFIG)

2.4 Frontend Technology Stack

HTML5

- Semantic markup for better structure and accessibility
- Form elements for user input
- Data attributes for client-side validation

CSS3

- Responsive design using Flexbox and Grid
- Media queries for mobile adaptability
- Modern styling for enhanced user experience
- Professional color schemes and typography

JavaScript (ES6+)

- Form validation and error handling
- Dynamic UI interactions
- Client-side ID proof validation
- AJAX for asynchronous operations

Jinja2 Template Engine

- Dynamic content generation
- Template inheritance for code reusability
- Conditional rendering based on user roles
- Loop constructs for displaying collections

UI Implementation Examples

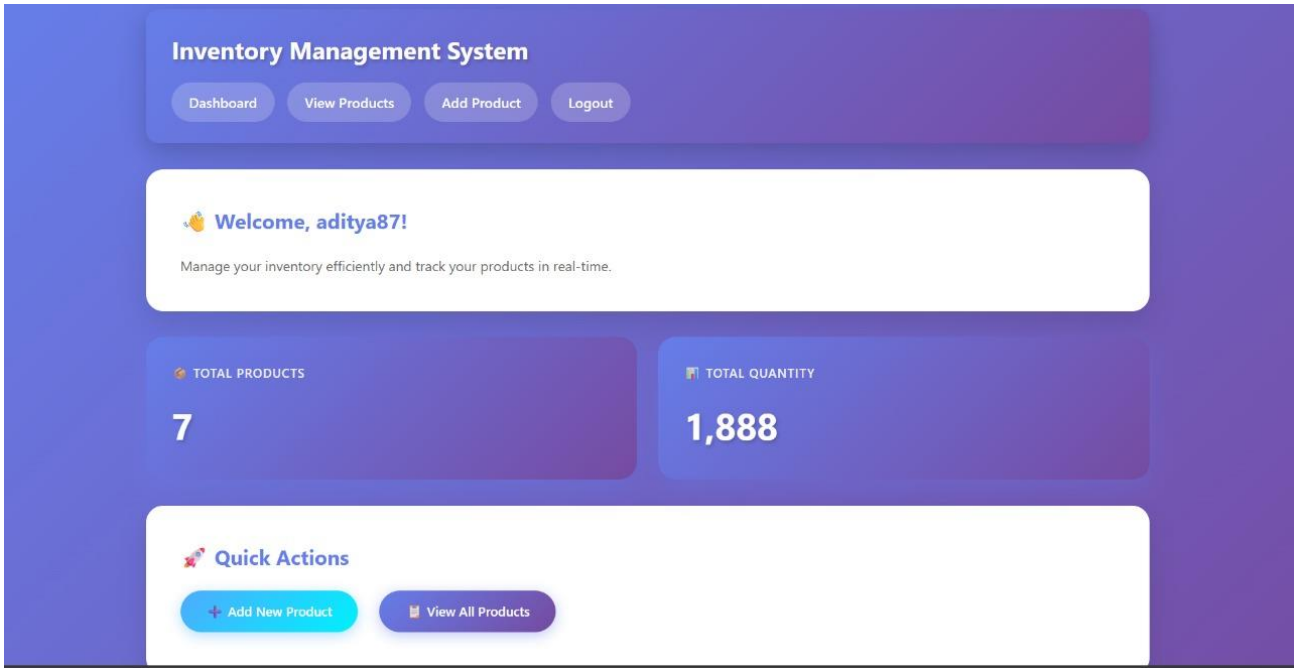

Figure 2.2: Inventory Dashboard

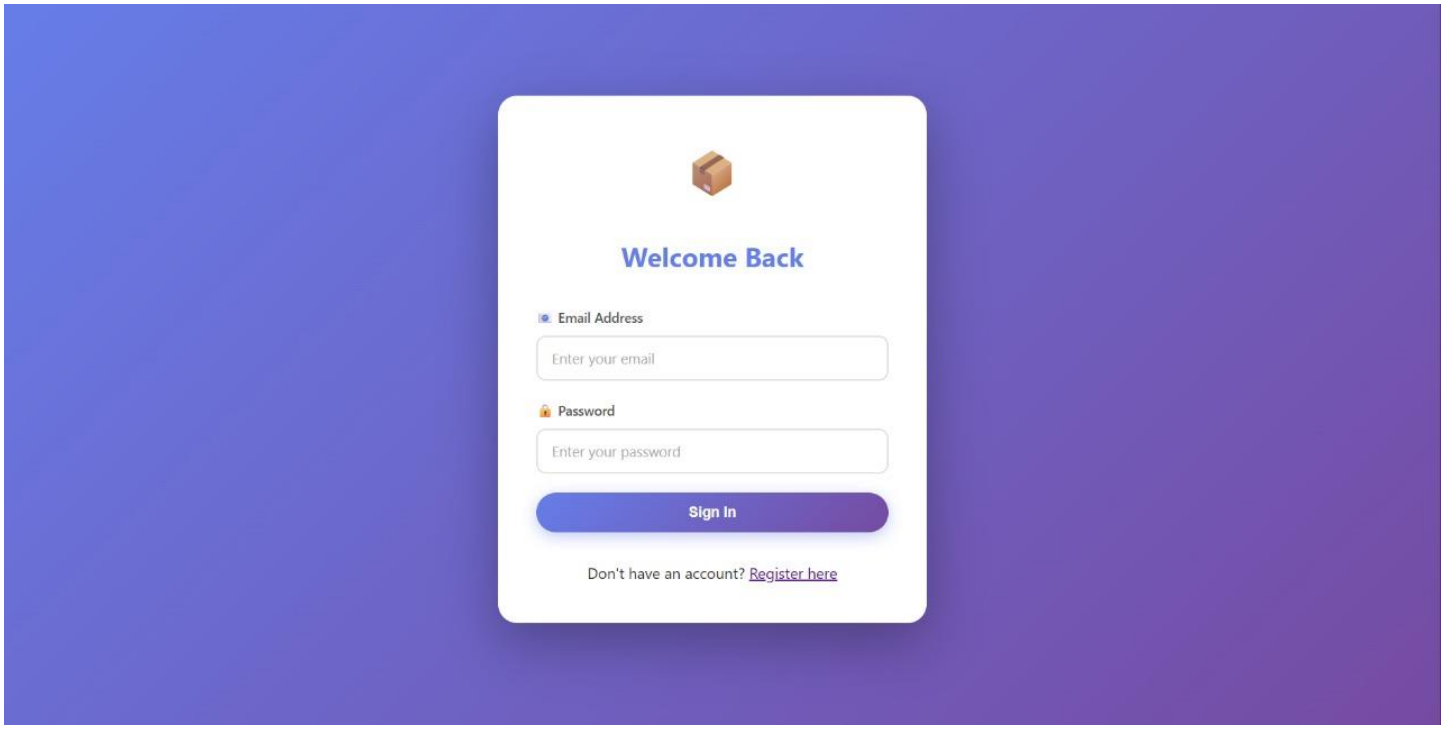

Fig2.3 : Login page

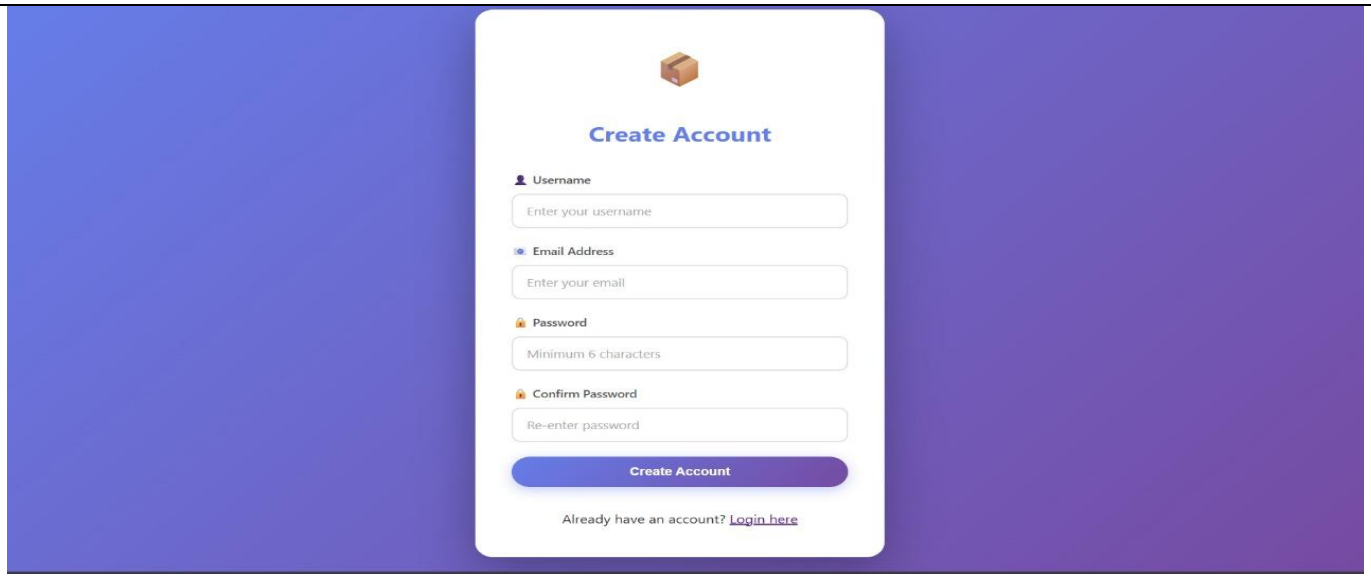

Figure 2.4:Sign in page

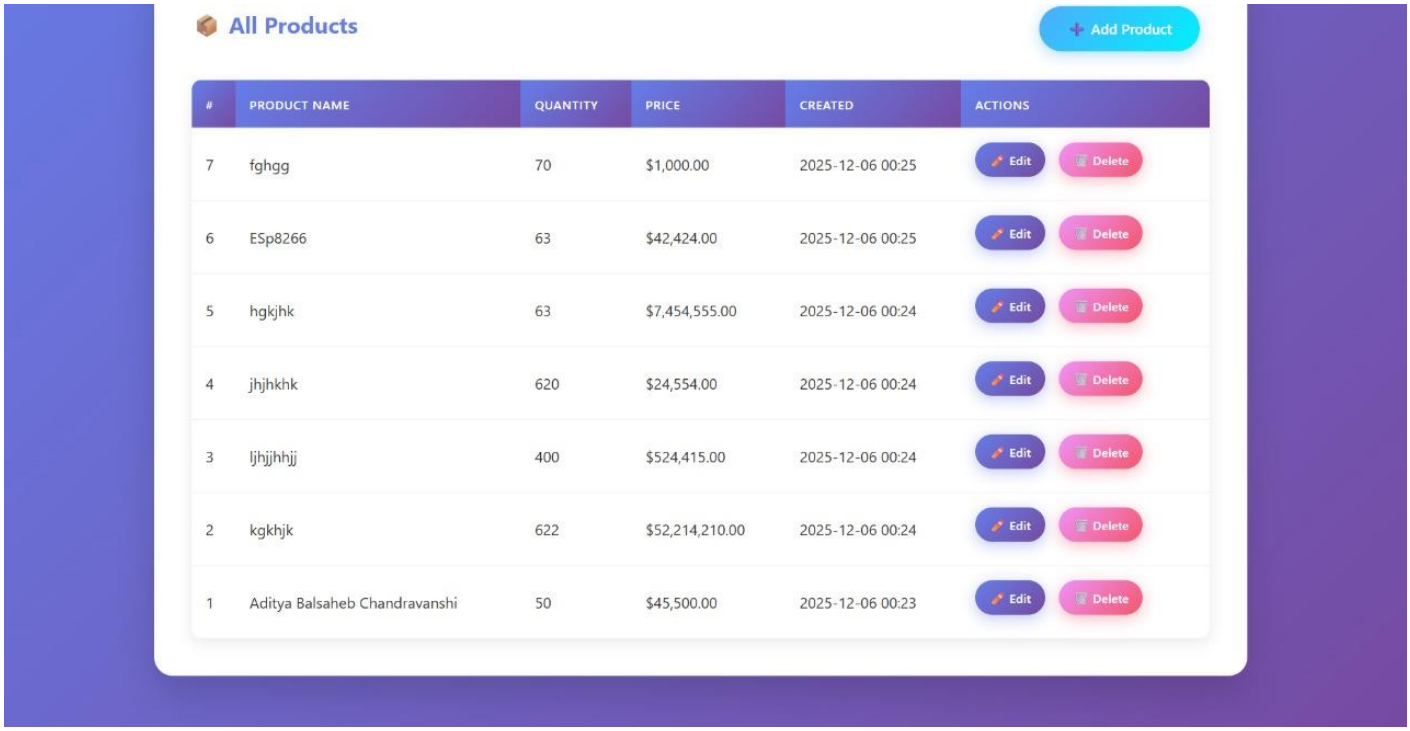

Figure 2.5: Review Application

2.5

Layer	Components
Presentation Layer	HTML, CSS, JavaScript, Jinja2 Templates
Business Logic Layer	Flask routes, functions, validations
Data Layer	MySQL database, PyMySQL connector

Table 2.1: Three-Tier Architecture of Inventory Management System

Request Flow:

1. User submits request through web interface
2. Flask application receives HTTP request
3. Application logic processes request
4. Database operations performed via PyMySQL
5. Response formatted in HTML/JSON
6. Response sent back to client
7. Browser renders updated interface

2.6 Development Environment

Development Stack:

- Python 3.8+
- Flask 2.3.3
- PyMySQL 1.1.0
- MySQL Server
- Text Editor/IDE
- Git for version control
- Browser (Chrome, Firefox, Safari, Edge)

2.7 System Requirements

Server Requirements:

- Minimum 2GB RAM

- 10GB disk space for database and files
- Linux/Windows/macOS operating system
- Python 3.8 or higher installed

Client Requirements:

- Modern web browser (2020+)
- JavaScript enabled
- Minimum 1MB internet connection
- Cookies enabled for session management

The Next chapter on Database Design and Normalization dives deeper into the MySQL schema, detailing entity-relationship models, 3NF implementation, and tables for voters, applications, and election cards to uphold data integrity and support efficient queries across the system

Chapter 3:

Database Design and Normalization

This chapter presents the design principles of the Inventory Management System's database, emphasizing normalization up to the Third Normal Form (3NF) to reduce data redundancy, enforce consistency, and improve query performance. It describes the schema of key tables users, voter applications, personal info, addresses, identifications, election cards, and update requests highlighting their roles and relationships through foreign keys to maintain integrity and support system workflows. The entity-relationship diagram illustrates the structured connections between users, applications, personal details, and related entities, while referential integrity and indexing strategies ensure efficient and reliable data operations.

3.1 Database Design Principles

The Inventory Management System employs a normalized database design following the Third Normal Form (3NF) principles [3][4]. Normalization is a systematic approach to organizing data to minimize redundancy and improve data integrity.

Normalization Benefits:

- **Eliminates Redundancy**: Data stored only once, reducing storage requirements
- **Maintains Consistency**: Changes made in one place automatically reflected everywhere
- **Improves Performance**: Optimized queries and faster data retrieval
- **Ensures Integrity**: Foreign keys maintain referential relationships
- **Facilitates Maintenance**: Easier to update and modify data structure

3.2 Normal Forms Applied

1NF - First Normal Form

- All values are atomic (non-divisible)
- No repeating groups
- Each field contains only one value per row

Example: Address fields separated into address_line, state, district, pincode rather than one combined address field.

2NF - Second Normal Form

- Satisfies 1NF requirements
- All non-key attributes depend on entire primary key
- Removal of partial dependencies

Example: Election card information separated from voter application table to avoid partial dependencies on application_id.

3NF - Third Normal Form

- Satisfies 2NF requirements
- No transitive dependencies between non-key attributes
- Each table focuses on specific entity

Example: Personal information, addresses, and identifications stored in separate tables rather than combined with applications.

Chapter **4**

Security Architecture and Implementation

This chapter introduces the comprehensive security measures implemented in the Inventory Management System to protect sensitive voter data and ensure system integrity. It covers key aspects like authentication, role-based access control, encryption, input validation, and protection against common web vulnerabilities such as SQL injection, XSS, and CSRF. Additionally, the chapter highlights session management, error handling, and audit trail mechanisms vital for robust security and compliance.

4.1 Security Framework

The Inventory Management System implements a comprehensive security framework protecting voter data and system integrity through multiple layers of protection [1][2][5]:

- Authentication mechanisms
- Authorization controls
- Data encryption
- Input validation
- Error handling
- Session management
- CSRF protection

4.2 Authentication System

Password Security

Hash Function: SHA-256

import hashlib

def hash_password(password): """Generate SHA-256 hash of password"""return hashlib.sha256(password.encode()).hexdigest()

def verify_password(stored_hash, password): """Verify password against stored hash""" return stored_hash == hashlib.sha256(password.encode()).hexdigest()

Password Requirements:

- Minimum 8 characters recommended
- Mix of uppercase, lowercase, numbers, and special characters
- Never stored in plain text
- Unique hashes for identical passwords (due to SHA-256 collision properties)

Login Process Workflow:

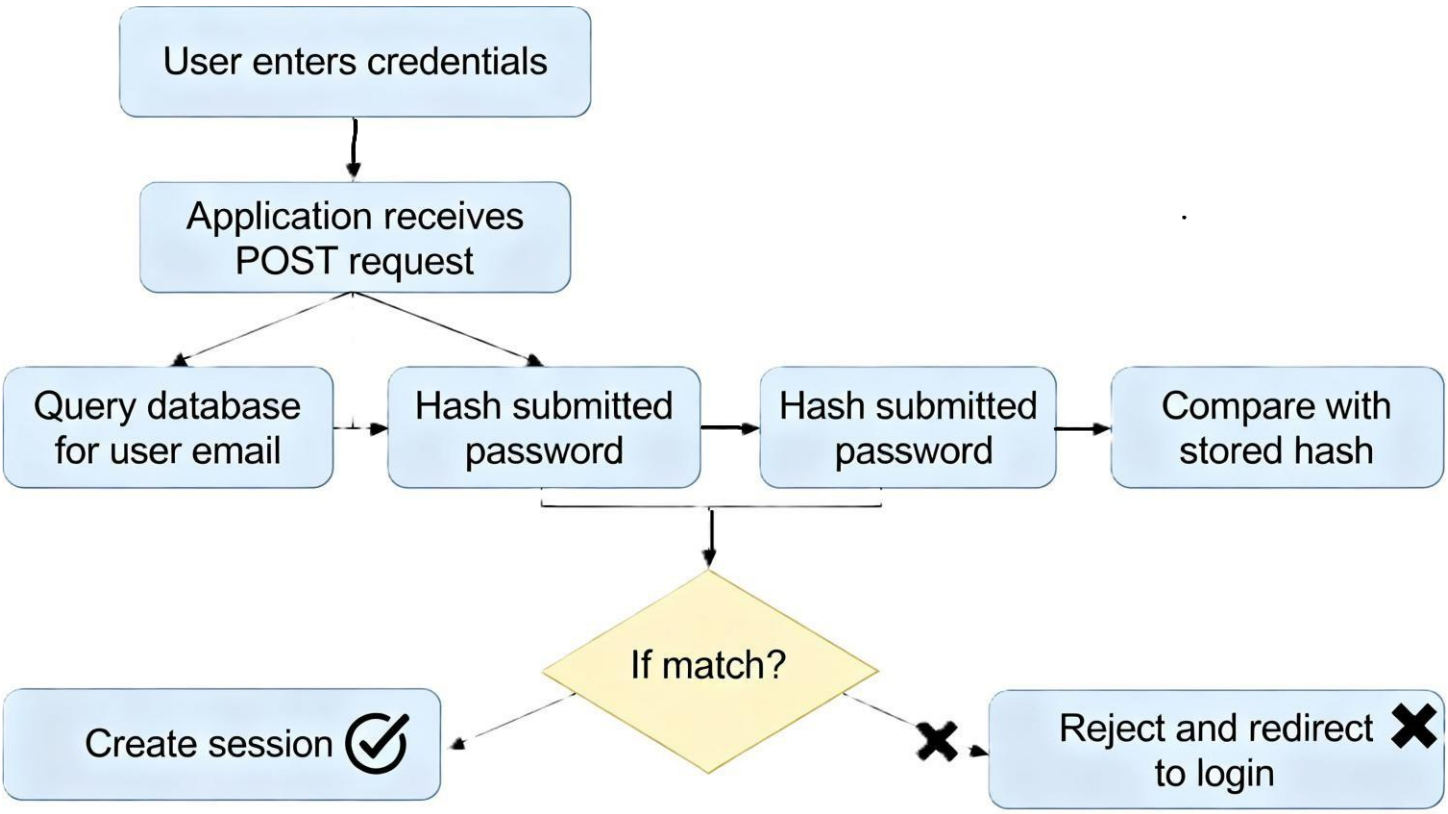

Figure 4.1: Authentication Session Workflow

CONCLUSION

The Inventory Management System successfully delivers a secure, scalable Flask-based web application for voter registration and administration, featuring a fully normalized MySQL database across seven interconnected tables that ensure data integrity and efficient querying. Key achievements include robust Role-Based Access Control (RBAC) implementation distinguishing voter and admin functionalities, comprehensive SHA-256 password hashing with session management, and multi-layered protections against SQL injection, XSS, and CSRF attacks following established Flask security best practices.

This system addresses core requirements for voter ID applications, status tracking, and administrative review while maintaining 3NF compliance to eliminate redundancy and support future scalability. The architecture supports high availability through persistent database connections and input validation at both client and server levels, making it production-ready for government or electoral commission deployment.

Future enhancements should prioritize email notifications, file upload capabilities for ID proofs, and RESTful API integration to enable mobile applications. The portal demonstrates practical application of modern web development principles, providing a solid foundation for electoral service digitization with measurable improvements in process efficiency and user trust.

REFERENCES

[1] Flask Community. (2024). *Protecting Flask Applications: Best Practices*. Retrieved from https://escape.tech/blog/best-practices-protect-flask-applications/

[2] Snyk Security. (2024). How to secure Python Flask applications. Retrieved from https://snyk.io/blog/secure-python-flask-applications/

[3] Real Python. (2024). Enhance Your Flask Web Project With a Database. Retrieved from https://realpython.com/flask-database/

[4] FreeCodeCamp. (2022). Normal Forms 1NF 2NF 3NF Table Examples. Retrieved from https://www.freecodecamp.org/news/database-normalization-1nf-2nf-3nf-table-examples/

[5] Sprinto. (2024). 5 Steps to Implementing Role Based Access Control (RBAC). Retrieved from https://sprinto.com/blog/how-to-implement-role-based-access-control/